When the Risk System Becomes the Risk: Design Principles for Assurance Integrity in Governance, Risk and Compliance Information Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. It has not asked whether the evidence they present can itself be trusted. A forensic audit of a deployed risk management system found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It then asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

Design/Methodology/Approach

The study follows design science research. A code and runtime audit of the deployed system produced a taxonomy of seven failure modes. Each failure mode was matched to a design principle grounded in established theory: fail-safe defaults, complete mediation, separation of duties, tamper-evident logging and the critique of ritualised assurance. The principles were instantiated in a new system, ERMA-GRC. The evaluation compared the original system with ERMA-GRC in 23 scenarios that were pre-registered before execution and run once each against identical stored data. Eighteen scenarios came from the audited failures, four tested the principles beyond them, and one was expected to fail.

Findings

The original system prevented the failure in 4 of the 18 audit-derived scenarios. ERMA-GRC prevented it in all 18. Each outcome was read from the stored data, not from what the system reported. It accepted no unauthenticated writes, rejected self-approval, recorded audit entries only under the signed-in identity, detected an edited audit entry, and returned errors instead of false successes. In the generalisation scenarios, ERMA-GRC prevented 3 of 4 failures and the original system none. Two failures of the new design are reported with equal weight. First, a key risk indicator's status was computed correctly in the user interface but could be declared by a client calling the server directly. Second, an insider holding the audit key could rewrite the evidence trail undetected after a restart. The results also show that the principles depend on one another. Separation of duties relies on the integrity of the audit trail, and a principle enforced only in the client offers no assurance.

Originality/Value

The study treats assurance integrity as a design property of GRC information systems rather than an organisational outcome. It contributes a failure taxonomy derived from a real system, seven design principles that form a nascent design theory, and a reusable, pre-registered scenario protocol. Practitioners and auditors can use the principles as a procurement and review checklist. The documented boundaries point to future work: externally anchored audit trails, server-side computation of every assurance metric, and evaluation with practitioners.

Keywords: assurance integrity; governance, risk and compliance; design science research; enterprise risk management; audit trail; segregation of duties

1. Introduction

Organisations increasingly record their risk management in GRC information systems. A risk register, a heatmap, an approved assessment and an audit trail are no longer working notes. They are the evidence that boards, external auditors and regulators rely on when they judge whether risks are understood and controlled (Power, 1997). The value of such a system therefore depends on more than its features. It depends on whether what it shows is true.

This study began with a forensic audit of a GRC system that was already deployed. The audit found that the software produced assurance evidence without any underlying record:

- ratings derived from row numbers;
- approvals that every subsequent save silently reset;
- an "immutable" audit trail kept in the browser, which anyone could rewrite;
- database failures reported to users as successful saves.

None of these were exotic security flaws. They were ordinary design decisions, and each one turned the system into a source of risk rather than a control over it.

The literature does not address this problem. GRC information systems research has defined the field and studied adoption and value (Papazafeiropoulou & Spanaki, 2016; Racz et al., 2010). Organisational research on enterprise risk management warns that assurance can decay into ritual (Power, 2009). The relevant mechanisms are well understood, but only in generic form (Saltzer & Schroeder, 1975; Schneier & Kelsey, 1999). No study we found connects application design failures to corrupted assurance evidence, or tests design principles against a real system.

This study addresses three research questions:

- **RQ1:** Which design failures in a GRC information system compromise the integrity of the assurance evidence it produces?
- **RQ2:** Which design principles, grounded in established kernel theories, address these failures?
- **RQ3:** To what extent does an instantiation of these principles prevent the failures, compared with the original system?

We answer them through design science research. The study contributes a failure taxonomy, seven design principles, and a pre-registered evaluation of the instantiated artifact, ERMA-GRC. Section 2 reviews the literature, Section 3 describes the method, and Section 4 reports and discusses the results. Sections 5 and 6 present the implications, limitations and future research.

2. Literature Review

2.1 Conceptual Background

Enterprise risk management (ERM) is the organisation-wide identification, assessment, treatment and monitoring of risk. *Governance, risk and compliance (GRC) information systems* integrate these activities and keep their records (Racz et al., 2010). *Assurance* is the confidence those records give boards, auditors and regulators that risks are known and controlled. We define *assurance integrity* as the property that every assurance artefact a GRC system presents is either a recorded, attributable action or an explicit statement that no such record exists. Such artefacts include a rating, an approval, an exposure figure and an audit entry. Assurance integrity is therefore a property of the system's design, not of the risks it manages.

2.2 Review of Previous Studies

ERM as practice.

- Organisational research shows that ERM is enacted, not simply adopted. Arena et al. (2010) trace how risk rationalities, experts and technologies combine differently across firms.
- Mikes (2009) shows that risk functions differ in their "calculative cultures".
- Mikes and Kaplan (2015) argue that a single ERM template is itself a risk.
- Meidell and Kaarbøe (2017) show that the tools an ERM function builds over time shape how much it influences decisions.
- Power (1997) warns that verification rituals can produce auditability rather than assurance. Power (2009) calls ERM at its weakest a "risk management of nothing".

Together these studies treat technology as part of how risk is produced. Its design is therefore a legitimate object of study.

GRC information systems.

- Racz et al. (2010) offered the first research frame for integrated GRC.
- Papazafeiropoulou and Spanaki (2016) interviewed GRC experts and derived a framework of implementation characteristics: goals, stakeholders, critical success factors and barriers.
- Spanaki and Papazafeiropoulou (2016) added an adoption lifecycle and value drivers.

This stream asks how GRC systems are adopted and what value they deliver. It does not ask whether the evidence they produce can be trusted.

Mechanisms (kernel theories). Each mechanism the artifact relies on is well established, but generically:

- Fail-safe defaults and complete mediation (Saltzer & Schroeder, 1975).
- Separation of duties in workflow systems (Botha & Eloff, 2001).
- Tamper-evident logging: hash chains (Schneier & Kelsey, 1999), and history trees whose integrity depends on external auditors holding commitments (Crosby & Wallach, 2009).
- Continuous assurance, which monitors controls through analytics (Vasarhelyi et al., 2004; Alles et al., 2006). It takes the monitoring system's own integrity for granted.
- Risk scoring: Cox (2008) shows that risk matrices can rank risks wrongly even when their inputs are correct.

Table 1 summarises the seven studies closest to this work.

Table 1. *Summary of selected literature*

Author(s)	Context	Methodology	Key Findings	Gap Identified
Racz et al. (2010)	Integrated GRC	Literature analysis, definition	A frame of reference for GRC research	Artifact design and evaluation
Papazafeiropoulou & Spanaki (2016)	GRC IS implementation	Literature review and expert interviews	Implementation characteristics and success factors	Integrity of the evidence the systems produce
Spanaki & Papazafeiropoulou (2016)	GRC IS adoption	Qualitative study	Adoption lifecycle; integration, optimisation and information as value drivers	Whether that information is trustworthy
Arena et al. (2010)	ERM in organisations	Comparative case study	ERM varies with its rationalities, experts and technologies	How a technology's design distorts risk information
Alles et al. (2006)	Continuous control monitoring	Pilot implementation (Siemens)	Business-process controls can be monitored continuously	Integrity of the monitoring system itself
Botha & Eloff (2001)	Workflow access control	Conceptual model	Static and dynamic separation-of-duties enforcement	Approval semantics of risk assessments
Crosby & Wallach (2009)	Secure logging	Data structure and analysis	Efficient tamper-evident logs, provided auditors hold commitments	Business workflows and other failure types

2.3 Research Gap

Within the literature reached by our documented search, no study examines how the design of a GRC application can itself corrupt the assurance evidence it presents:

- by fabricating values;
- by overwriting decisions;
- by accepting unauthenticated authority;
- by reporting failures as success.

The two relevant streams do not meet. GRC information systems research stops at adoption and value. Mechanism research does not address the evidence that risk and control self-assessment produces: ratings, approvals, appetite exposure and indicator status.

The gap is therefore not a missing mechanism, which we do not claim. It is missing **design knowledge** that maps application failures to corrupted evidence, and that tests principles against a real system.

2.4 Research Framework

The study links four elements in a chain:

1. **Failure modes** observed in a deployed system (RQ1).
2. **Design principles**, each grounded in a kernel theory above (RQ2).
3. **Mechanisms** that instantiate each principle.
4. **Scenario outcomes** that test whether the failures are prevented (RQ3).

The chain follows the structure of a design theory (Gregor & Jones, 2007). The principles take the aim–mechanism–rationale form proposed by Gregor et al. (2020). Table 2 (Section 4) makes the mapping explicit.

3. Methodology

3.1 Research Design

We followed design science research (Hevner et al., 2004) and the six activities of the DSRM (Peffers et al., 2007):

1. **Problem identification.** A forensic audit of a GRC/ERM system that was already deployed. The audit combined code review, runtime reproduction and data-path tracing, and produced seven failure modes (RQ1).
2. **Design.** We expressed remedies as design principles in the aim–mechanism–rationale form of Gregor et al. (2020), each grounded in a kernel theory (RQ2).
3. **Instantiation.** The principles were instantiated in ERMA-GRC, a web application whose server mediates every read and write to an Oracle database.
4. **Evaluation (RQ3).** We followed the *technical risk and efficacy* strategy of FEDS (Venable et al., 2016): artificial, summative and ex post.

3.2 Data Collection

Data came from the audit of the original system (code, runtime behaviour and data paths) and from a scenario experiment whose protocol a timestamped repository commit recorded before any run.

Systems compared:

- **Baseline:** the original system at its last commit before the redesign.
- **Artifact:** ERMA-GRC.

Both ran against the same in-memory stand-in for the database interface, seeded with identical rows. Ground truth was read from the stand-in, never from the system under test (Figure 1). Each scenario ran once per system. Evidence (HTTP status, stored row, screen text, screenshot) was kept for every run.

3.3 Measures and Instruments

The unit of analysis is a scenario run. The experiment contains 23 scenarios in three groups. Eighteen are audit-derived: each operationalises one failure mode. Four generalisation scenarios test the principles beyond the audited defects. One boundary scenario, expected to fail, records a known limitation. Each outcome was coded against a criterion fixed in advance: PREVENTED, OCCURRED, or not applicable.

3.4 Data Analysis

- For each failure mode and system, we count PREVENTED scenarios out of the runnable ones.
- We use no inferential statistics. The outcomes are deterministic, and the scenarios are a test suite, not a sample.
- Protocol clarifications and one environmental re-run are reported in the replication package.

Use of generative AI. Claude (Claude Opus 5.5, Anthropic) was used under the authors' direction to:

- (a) assist the code and runtime audit of the baseline;

- (b) implement substantial parts of ERMA-GRC;
- (c) script the evaluation scenarios;
- (d) support the literature search and DOI verification;
- (e) draft text, which the authors revised.

The authors designed the study, defined the principles and scenarios, verified all sources and results, and take full responsibility for the content. No AI tool is listed as an author.

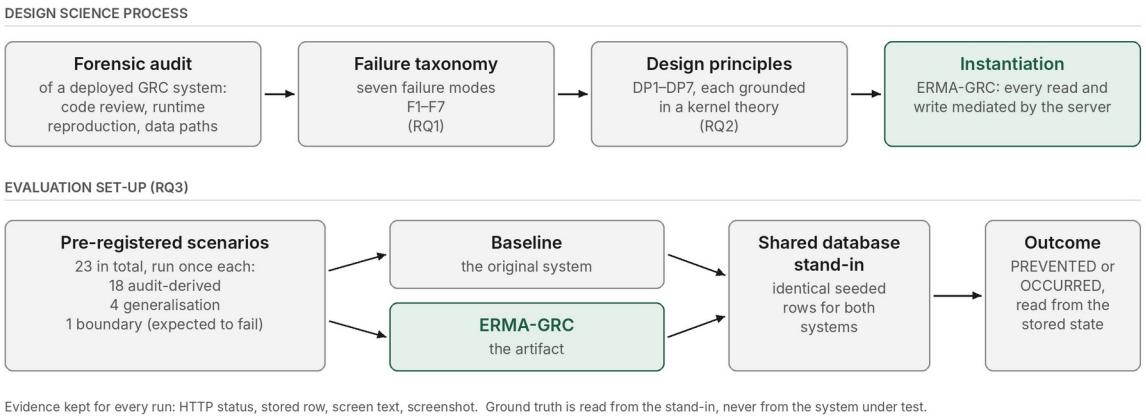


Figure 1. DSR process and evaluation set-up. The audit of the baseline yields F1–F7, which inform DP1–DP7, which are instantiated in ERMA-GRC. The baseline and ERMA-GRC each run the 23 pre-registered scenarios against one shared database stand-in, and outcomes are coded against its stored state.

4. Results and Discussion

4.1 Results

RQ1 and RQ2. Table 2 maps the seven failure modes to the principles that counter them and to the mechanisms that instantiate those principles in ERMA-GRC.

Table 2. *Failure modes, design principles and mechanisms*

Failure mode (RQ1)	Design principle (RQ2)	Mechanism in ERMA-GRC
F1 Fabricated assurance values	DP1 Evidence provenance	No derived ratings; unrated risks shown as "Not rated" and kept off the heatmap
F2 Decisions overwritten by defaults	DP2 Decision preservation	Defaults fill only missing fields at creation; approval fields change only through authorised transitions
F3 Client-side or absent enforcement	DP3 Server-enforced, identity-bound separation of duties	Signed session; deny-by-default permission matrix; maker ≠ checker; conditional writes (ETag) on decisions; deactivation revokes sessions
F4 Mutable, forgeable evidence trail	DP4 Tamper-evident, actor-bound trail	Append-only HMAC-SHA256 hash chain; actor taken from the session only; verification shown to users
F5 Silent failure reported as success	DP5 Fail-visible operation	Database failures returned as explicit error statuses; no fallback store
F6 Fragmented system of record	DP6 Single system of record	All modules persisted in the database

Failure mode (RQ1)	Design principle (RQ2)	Mechanism in ERMA-GRC
F7 Destructive actions on fabricated records	DP1 Evidence provenance	Empty states instead of placeholder rows
(F1)	DP7 Computed, not declared, metrics	KRI status derived from thresholds and direction

RQ3. Table 3 summarises the outcomes:

- On the 18 audit-derived scenarios, ERMA-GRC prevented 18 failures and the baseline prevented 4.
- On the generalisation scenarios, ERMA-GRC prevented 3 of 4 and the baseline none.
- The boundary scenario occurred on both systems, as registered.

The baseline's failures were concrete:

- It stored DRAFT after an approver set APPROVED.
- It accepted writes carrying no session, or only a self-declared administrator header.
- It recorded an audit entry under a claimed "CEO" identity.
- It answered 201 while the database was unreachable.
- It invented two scored risks, with delete buttons, for an assessment cycle that had none.

In the generalisation scenarios, ERMA-GRC:

- kept exactly one of two simultaneous approval decisions, and the other approver received a conflict;
- rejected a write made with the still-valid session of a deactivated user;
- rejected a path-traversal identifier before it reached the database.

The baseline reported both simultaneous decisions as successful, accepted the write, and forwarded the traversal to the database.

Table 3. *Scenario outcomes (PREVENTED / runnable)*

Scenarios	Baseline	ERMA-GRC
F1 Fabricated values (S1.1–S1.4)	3 / 4	4 / 4
F2 Decisions overwritten (S2.1–S2.2)	0 / 2	2 / 2
F3 Enforcement (S3.1–S3.4)	0 / 4	4 / 4
F4 Evidence trail (S4.1–S4.3)	1 / 3	3 / 3
F5 Silent failure (S5.1–S5.3)	0 / 3	3 / 3
F6 Fragmented record (S6.1)	0 / 1	1 / 1
F7 Fabricated-row deletion (S7.1)	0 / 1	1 / 1
Generalisation (G1–G4)	0 / 4	3 / 4
Boundary (L1, expected to fail)	0 / 1	0 / 1
Total	**4 / 23**	**21 / 23**

Where the baseline passed. The baseline met four criteria:

- It displayed stored statements, monetary values and empty statuses faithfully.
- It attributed an audit entry to the acting persona, though that identity was self-asserted.
- On a fresh installation its audit trail held no invented entries. This contradicts the audit report: the seeding code exists, but it never runs on that path.

Where ERMA-GRC failed. It failed twice:

- **G4.** KRI status was computed correctly through the interface, but the API stored a status declared by the client. DP7 was therefore enforced only in the browser. The baseline behaved the same way.

- **L1.** An insider holding the audit key re-chained an edited log offline. After a restart, verification passed.

Figure 2 shows two of the baseline's failures next to ERMA-GRC's handling of the same stored data.

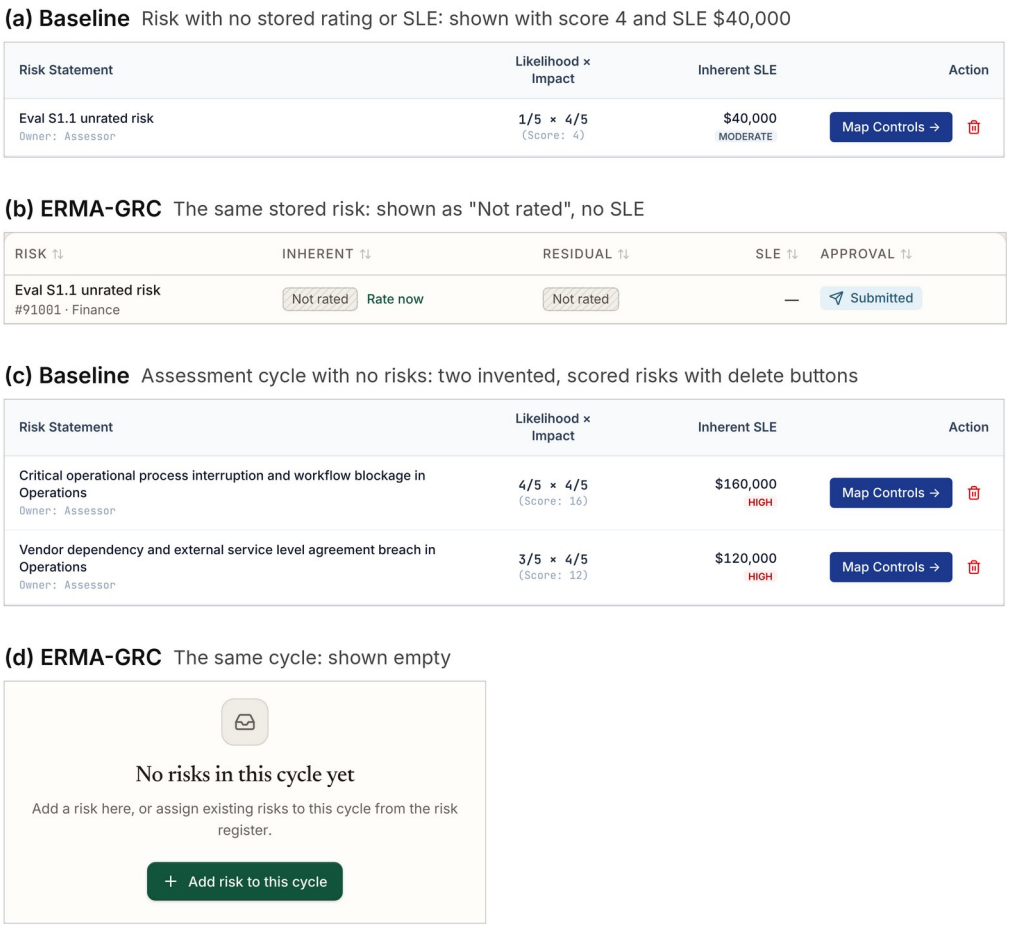


Figure 2. The same stored data in both systems. (a) The baseline shows a risk with no stored rating or SLE with a score of 4 and an SLE of $40,000 (scenario S1.1). (b) ERMA-GRC shows the same risk as "Not rated". (c) For an assessment cycle that has no risks, the baseline shows two invented, scored risks, each with a delete button (S7.1). (d) ERMA-GRC shows the cycle as empty. Screens were re-captured after the product was renamed, from the same set-up as the recorded run; table columns unrelated to the comparison are hidden.

4.2 Discussion

The failures are assurance failures. The baseline produced scores, approvals and audit entries that were disconnected from its stored state: a "risk management of nothing" (Power, 2009) produced by software rather than by organisational ritual. Several mechanisms are generic: fail-safe defaults and complete mediation (Saltzer & Schroeder, 1975), separation of duties (Botha & Eloff, 2001) and hash-chained logs (Schneier & Kelsey, 1999). What is specific to GRC is the object they protect. DP1, DP2 and DP7 have no direct counterpart in generic security guidance, because they concern whether a displayed rating or decision is evidence at all.

The principles depend on each other. ERMA-GRC identifies the creator of a record, and so who may not approve it, from its audit trail. In L1, rewriting one creation entry changed who was barred from approving that record. DP3 therefore rests on DP4, and DP4 rests on keys held

outside the application. This is the boundary Crosby and Wallach (2009) predict for logs without external auditors.

Where a principle is enforced matters. G4 shows that a principle checked only in the client is a convention, not a control. We therefore refine the design theory: every principle must be enforced at the server boundary.

Assurance integrity remains necessary but not sufficient. Faithful records of a poorly calibrated risk matrix are still poor risk information (Cox, 2008).

5. Implications

Theoretical implications.

- The study recasts assurance integrity as a property of GRC system design. This adds a design perspective to research that has so far treated GRC systems through adoption and value.
- The principles are not independent. Separation of duties depends on the integrity of the evidence trail. Every principle holds only if it is enforced at the server.
- These relationships give the nascent design theory testable propositions.

Practical implications. GRC buyers, internal auditors and external auditors can turn the principles into review questions:

- Does the system show an unrated risk as unrated?
- Can an ordinary save overwrite an approval?
- Is authority taken from a verified session, or from what the client claims?
- Can users verify the audit trail, and is it anchored outside the vendor's control?
- Does a failed write ever appear as a success?

These questions can be tested in a demonstration before purchase, much as the scenarios here were.

Societal implications. Regulators and the public increasingly rely on compliance claims produced by software. Systems that cannot fabricate evidence make those claims more trustworthy. They also make accountability harder to evade.

6. Conclusion, Limitations and Future Research

Conclusion. This study asked how the design of a GRC information system can corrupt the assurance evidence it produces, and how to prevent it.

- An audit of a deployed system yielded seven failure modes.
- Seven principles, grounded in established theory, address those failure modes.
- ERMA-GRC, which instantiates the principles, prevented every audit-derived failure. The original system prevented four of eighteen.
- The two failures of the new design matter as much as its successes: a metric enforced only in the client, and an audit trail its key holder can rewrite.

Limitations.

- **Single case.** The evidence comes from one case, and the authors built the artifact and wrote the scenarios. Pre-registration and reporting every outcome reduce this bias but do not remove it.
- **Artificial evaluation.** The evaluation ran against a database stand-in, not against practitioners. Database-level claims rest on a separate install on Oracle Database 23ai.

- **Generative AI.** It assisted the audit, the implementation and the drafting, as disclosed in Section 3.
- **Literature coverage.** The literature search could not use Scopus or Web of Science.
- **No comparison with commercial products.** The study makes no claim about commercial GRC products.

Future research.

- Naturalistic evaluation with risk and audit professionals.
- Multiple-case studies across GRC products.
- Externally anchored audit commitments.
- Server-side computation of every assurance metric.
- Longitudinal study of whether integrity-preserving systems change how organisations use risk information.

References

Alles, M., Brennan, G., Kogan, A., & Vasarhelyi, M. A. (2006). Continuous monitoring of business process controls: A pilot implementation of a continuous auditing system at Siemens. *International Journal of Accounting Information Systems, 7*(2), 137–161. https://doi.org/10.1016/j.accinf.2005.10.004

Arena, M., Arnaboldi, M., & Azzone, G. (2010). The organizational dynamics of enterprise risk management. *Accounting, Organizations and Society, 35*(7), 659–675. https://doi.org/10.1016/j.aos.2010.07.003

Botha, R. A., & Eloff, J. H. P. (2001). Separation of duties for access control enforcement in workflow environments. *IBM Systems Journal, 40*(3), 666–682. https://doi.org/10.1147/sj.403.0666

Cox, L. A., Jr. (2008). What's wrong with risk matrices? *Risk Analysis, 28*(2), 497–512. https://doi.org/10.1111/j.1539-6924.2008.01030.x

Crosby, S. A., & Wallach, D. S. (2009). Efficient data structures for tamper-evident logging. In *Proceedings of the 18th USENIX Security Symposium* (pp. 317–334). USENIX Association.

Gregor, S., Chandra Kruse, L., & Seidel, S. (2020). Research perspectives: The anatomy of a design principle. *Journal of the Association for Information Systems, 21*(6), 1622–1652. https://doi.org/10.17705/1jais.00649

Gregor, S., & Jones, D. (2007). The anatomy of a design theory. *Journal of the Association for Information Systems, 8*(5), 312–335. https://doi.org/10.17705/1jais.00129

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–106. https://doi.org/10.2307/25148625

Meidell, A., & Kaarbøe, K. (2017). How the enterprise risk management function influences decision-making in the organization – A field study of a large, global oil and gas company. *The British Accounting Review, 49*(1), 39–55. https://doi.org/10.1016/j.bar.2016.10.005

Mikes, A., & Kaplan, R. S. (2015). When one size doesn't fit all: Evolving directions in the research and practice of enterprise risk management. *Journal of Applied Corporate Finance, 27*(1), 37–40. https://doi.org/10.1111/jacf.12102

Mikes, A. (2009). Risk management and calculative cultures. *Management Accounting Research, 20*(1), 18–40. https://doi.org/10.1016/j.mar.2008.10.005

Papazafeiropoulou, A., & Spanaki, K. (2016). Understanding governance, risk and compliance information systems (GRC IS): The experts view. *Information Systems Frontiers, 18*(6), 1251–1263. https://doi.org/10.1007/s10796-015-9572-3

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Power, M. (1997). *The audit society: Rituals of verification*. Oxford University Press.

Power, M. (2009). The risk management of nothing. *Accounting, Organizations and Society, 34*(6–7), 849–855. https://doi.org/10.1016/j.aos.2009.06.001

Racz, N., Weippl, E., & Seufert, A. (2010). A frame of reference for research of integrated governance, risk and compliance (GRC). In B. De Decker & I. Schaumüller-Bichl (Eds.), *Communications and Multimedia Security* (Lecture Notes in Computer Science, Vol. 6109, pp. 106–117). Springer. https://doi.org/10.1007/978-3-642-13241-4_11

Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE, 63*(9), 1278–1308. https://doi.org/10.1109/PROC.1975.9939

Schneier, B., & Kelsey, J. (1999). Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security, 2*(2), 159–176. https://doi.org/10.1145/317087.317089

Spanaki, K., & Papazafeiropoulou, A. (2016). The implementation of governance, risk, and compliance IS: Adoption lifecycle and enterprise value. *Information Systems Management, 33*(4), 302–315. https://doi.org/10.1080/10580530.2016.1220214

Vasarhelyi, M. A., Alles, M. G., & Kogan, A. (2004). Principles of analytic monitoring for continuous assurance. *Journal of Emerging Technologies in Accounting, 1*(1), 1–21. https://doi.org/10.2308/jeta.2004.1.1.1

Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: A framework for evaluation in design science research. *European Journal of Information Systems, 25*(1), 77–89. https://doi.org/10.1057/ejis.2014.36